

Chapter-4

SQL COMMANDS



➤ What is SQL?

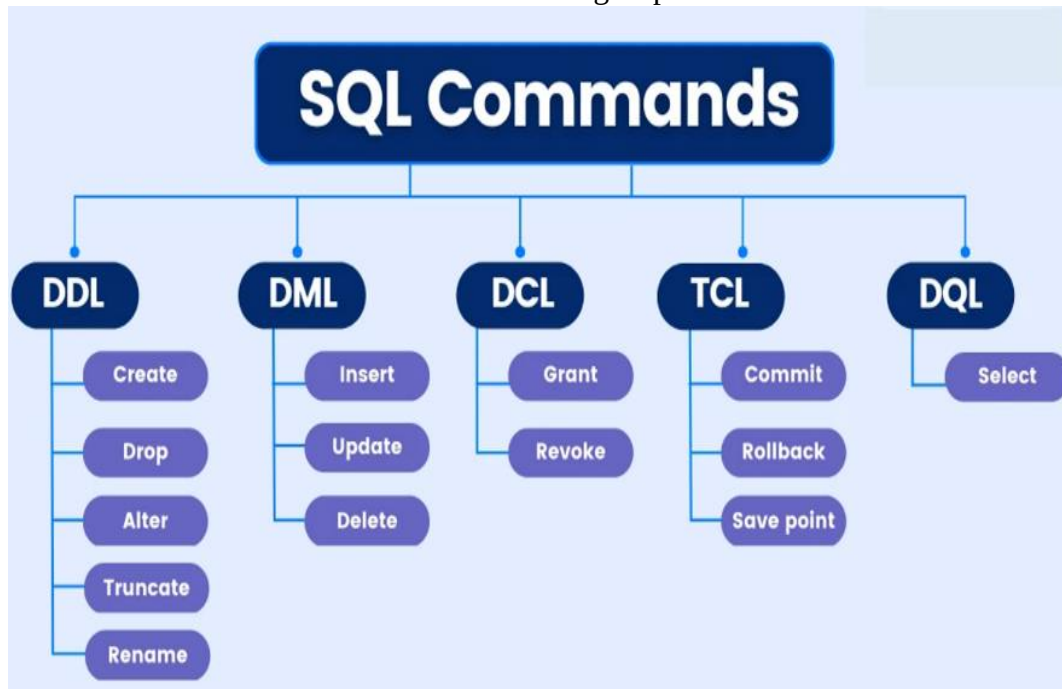
- **Structured Query Language** and it helps to make practice on SQL commands which provides immediate results.
- SQL is Structured Query Language, which is a computer language for storing, manipulating and retrieving data stored in relational database.
- SQL is the standard language for Relation Database System.
- All relational database management systems like MySQL, MS Access, and Oracle, Sybase, Informix, and SQL Server use SQL as standard database language.

➤ Why SQL?

- Allow users to create and drop databases and tables.
- Allow users to describe the data.
- Allows users to define the data in database and manipulate that data.
- Allow users to access data in relational database management systems.
- Allows embedding within other languages using SQL modules, libraries & pre-compilers.
- Allows users to set permissions on tables, procedures, and views

➤ SQL Commands:

- The standard SQL commands to interact with relational databases are CREATE, SELECT, INSERT, UPDATE, DELETE and DROP.
- These commands can be classified into groups based on their nature:



➤ DDL - Data Definition Language:

- *It provides a link between the logical schema and the physical storage of data.*

Functions of DDL:

1. Defines the structure of database objects such as tables, fields/columns, data types, field length, constraints, and relationships.
2. Describes the schema and subschema of the database.
3. Specifies keys such as primary key, foreign key, and unique key.
4. Provides logical and physical data independence by allowing structure changes without affecting applications.

Basic DDL Commands

Command	Description
CREATE	Creates a new table, a view of a table, or other object in database
ALTER	Modifies an existing database object, such as a table.
DROP	Delete an entire table, a view of a table or other object in the database.
Truncate	Removes all records from a table permanently but it does not delete the table itself.
Rename	Change the name of a table or other database object

➤ **DML - Data Manipulation Language:**

- *DML provides data manipulation techniques like insertion, deletion, updating, modification, replacement, retrieval, sorting and display of data or records.*
- DML facilitates use of relationship between the records.
- DML provides for independence of programming languages by supporting several high-level programming languages like COBOL, PL/1 and C++.
- Few of the basic commands for DML are:

Command	Description
INSERT	Creates a record
UPDATE	Modifies records
DELETE	Deletes records

➤ DCL - Data Control Language:

- These SQL commands are used for providing security to database objects.
- The different DCL commands are:

Command	Description
GRANT	Gives a privilege to user
REVOKE	Takes back privileges granted from user

➤ TCL – Transaction Control Language:

- It includes commands to control the transactions in a database system.
- The commonly used commands are:

Command	Description
COMMIT	Make all the changes made by the statements issued permanent.
ROLLBACK	Undoes all change since the beginning of transaction or since a save point.

Data Types in SQL:**Data Types:**

- A data type defines what kind of values a column can store in a table.
(Example: numbers, text, dates, etc.)
- Data types are specified while creating a table.
- SQL data types are mainly classified into three commonly used categories:

1.String / Character Data Types

Used to store text values.

Examples:

- CHAR
- VARCHAR
- TEXT
- NCHAR
- NVARCHAR

2. Numeric Data Types

Used to store numbers.

Examples:

- INT
- FLOAT
- DOUBLE
- DECIMAL
- BOOLEAN (BOOL) stores TRUE/FALSE

3.Date and Time Data Types

Used to store date and time values.

Examples:

- DATE
- TIME
- DATETIME

➤ What is an Operator in SQL?

An operator is a reserved word or symbol used in SQL to perform operations on data, often in the WHERE clause or in expressions.

Operators are used to:

- Specify conditions in queries.
- Perform arithmetic, comparison, or logical operations.
- Combine multiple conditions in a query.

Types of SQL Operators

1.Arithmetic Operators

Used for numeric calculations:

Operator	Meaning
+	Addition
-	Subtraction
*	Multiplication
/	Division
%	Modulus (remainder)

2. Comparison Operators

Used to compare values:

Operator	Meaning
=	Equal to
!= or <>	Not equal to
>	Greater than
<	Less than
>=	Greater than or equal to
<=	Less than or equal to
!<	Not less than
!>	Not greater than

3.Logical / Conditional Operators

Used to combine or test multiple conditions:

Operator	Meaning
AND	Both conditions must be true
OR	At least one condition is true
NOT	Negates a condition

➤ Implementation of SQL Commands

1. CREATE TABLE

- The SQL **CREATE TABLE** statement is used to create a new table.
- Creating a basic table involves naming the table and defining its columns and each column's data type.
- **Syntax:** Basic syntax of **CREATE TABLE** statement is as follows:

```
CREATE TABLE Table_name
(
    column1 datatype,
    column2 datatype,
    column3 datatype,
    ....
    columnN datatype,
    PRIMARY KEY( one or more columns )
);
```

- Here CREATE TABLE is the keyword followed by the Table_name, followed by an open parenthesis, followed by the column names and data types for that column, and followed by a closed parenthesis.
- For each column, a name and a data type must be specified and the column name must be a unique within the table definition.
- Column definitions are separated by commas (,).
- Uppercase and lowercase letters makes no difference in column names.
- Each table must have at least one column.
- SQL commands should end with a semicolon (;).

- Example: Create a table “STUDENT” that contains five columns: RegNo, Name, Combination, DOB and Fees.

```
CREATE TABLE STUDENT
(
    RegNo          NUMBER (6),
    Name           VARCHAR(15),
    Combination     CHAR (4),
    DOB            DATE,
    Fees           NUMBER (9, 2),
    PRIMARY KEY ( RegNo )
);
```

It creates an empty STUDENT table which looks like this:

RegNo	Name	Combination	DOB	Fees
-------	------	-------------	-----	------

2. ALTER Statement:

- The table can be modified or changed by using the ALTER command.
- Syntax: Basic syntax of ALTER TABLE statement is as follows:

1. ALTER TABLE Table_name
 ADD (column_name1 DataType, Cloumn_name2 DataType.....);
2. ALTER TABLE Table_name
 MODIFY (column_name1 DataType, Cloumn_name2 DataType.....);
3. ALTER TABLE Table_name
 DROP (column_name1 DataType, Cloumn_name2 DataType.....);

- Example:

```
SQL> ALTER TABLE STUDENT ADD (Address VARCHAR2 (30));
Table altered.

SQL> ALTER TABLE STUDENT MODIFY (Address VARCHAR2 (40));
Table altered.

SQL> ALTER TABLE STUDENT DROP (ADDRESS);
Table altered.
```

3. DROP TABLE:

- The SQL **DROP TABLE** statement is used to remove a table definition and all data, indexes, triggers, constraints, and permission specifications for that table.
- Syntax: Basic syntax of **DROP TABLE** statement is as follows:

```
DROP TABLE Table_name;
```

- Example:

```
SQL> DROP TABLE STUDENT;  
Table dropped.
```

4. Truncate:

- Truncate Table** statement is used to remove all rows (complete data) from a table.
- Truncate Table vs Drop Table:** Drop table command can also be used to delete complete table but it deletes table structure too. Truncate Table doesn't delete the structure of the table.
- Syntax:** Truncate Table table_name;

Emp

Emp_ID	Name	Age	Location	Salary
1	Ram	32	Ktm	20000
2	Hari	32	Mnr	20000

Example: Truncate Table Emp;

Emp

Emp_ID	Name	Age	Location	Salary

5. RENAME:

- RENAME TABLE statement is used to change the name of a table.
- Syntax: Rename Table old_table_name TO New_Table_Name;
Example: Rename Table Emp To Employee;

Syntax: RENAME TABLE old_table_name TO new_table_name;

DML COMMANDS:

✓ INSERT:

- The SQL **INSERT INTO** Statement is used to add new rows of data to a table in the database.
- Syntax:
- There are two basic syntaxes of INSERT INTO statement as follows:

```
INSERT INTO TABLE_NAME [(column1, column2, column3,...columnN)]  
VALUES (value1, value2, value3,...valueN);
```

- Here, column1, column2,...columnN are the names of the columns in the table into which you want to insert data.
- You may not need to specify the column(s) name in the SQL query if you are adding values for all the columns of the table. But make sure the order of the values is in the same order as the columns in the table.
- **METHOD 1:** The SQL INSERT INTO syntax would be as follows:

```
INSERT INTO TABLE_NAME VALUES (value1,value2,value3,...valueN);
```

- Example: Following statements would create six records in STUDENT table:

```
SQL> INSERT INTO STUDENT VALUES(1401,'RAMESH','PCMC','07-AUG-99',14000);
1 row created.
SQL> INSERT INTO STUDENT VALUES(1402,'JOHN','PCMB','15-SEP-99',13500);
1 row created.
SQL> INSERT INTO STUDENT VALUES(1403,'GANESH','PCME','19-AUG-99',16000);
1 row created.
SQL> INSERT INTO STUDENT VALUES(1404,'MAHESH','PCMC','14-JAN-98',17650);
1 row created.
SQL> INSERT INTO STUDENT VALUES(1405,'SURESH','PCMB','03-MAR-98',11500);
1 row created.
SQL> INSERT INTO STUDENT VALUES(1410,'ARUN','PCMC','01-APR-04',13000);
```

- **METHOD 2:** The SQL INSERT INTO syntax would be as follows:

```
SQL> INSERT INTO STUDENT (REGNO, NAME, FEES) VALUES (1411, 'SHREYA',24000);
1 row created.
```

```
SQL> INSERT INTO STUDENT (REGNO, COMBINATION,FEES) VALUES(1412,
'PCMB',21000);
1 row created.
```

- All the above statements would produce the following records in STUDENT table:

```
SQL> SELECT * FROM STUDENT;
```

REGNO	NAME	COMB	DOB	FEES
1401	RAMESH	PCMC	07-AUG-99	14000
1402	JOHN	PCMB	15-SEP-99	13500
1403	GANESH	PCME	19-AUG-99	16000
1404	MAHESH	PCMC	14-JAN-98	17650
1405	SURESH	PCMB	03-MAR-98	11500
1410	ARUN	PCMC	01-APR-04	13000
1411	SHREYA			24000
1412		PCMB		21000

```
8 rows selected.
```

✓ UPDATE:

- SQL provides the ability to change data through UPDATE command.
- The UPDATE command used to modify or update an already existing row or rows of a table.
- The basic syntax of UPDATE command is given below.

```
UPDATE      Table_name
SET         column_name = value
           [, column_name =value..... ]
[WHERE      condition];
```

Example:

```
SQL> UPDATE STUDENT SET COMBINATION='CEBA' WHERE REGNO=1411;
1 row updated.
SQL> UPDATE STUDENT SET NAME='AKASH' WHERE REGNO=1412;
1 row updated.
```

✓ DELETE command:

- In SQL, an already existing row or rows are removed from tables through the use of DELETE command.
- The basic syntax of DELETE command is given below.

```
DELETE      Table_name
[WHERE      condition];
```

Example:

```
SQL> DELETE STUDENT WHERE REGNO=1412;
1 row deleted.
```

DQL COMMAND: SELECT:

- SQL **SELECT** statement is used to fetch the data from a database table which returns data in the form of result table. These result tables are called result-sets.
- Syntax: The basic syntax of SELECT statement is as follows:

SELECT column1, column2, columnN FROM Table_name;	Compulsory Part
[WHERE condition(s)] [GROUPBY column-list] [HAVING condition(s)] [ORDER BY column-name(s)];	Optional Part

- Here, column1, column2...are the fields of a table whose values you want to fetch. If you want to fetch all the fields available in the field, then you can use the following syntax:

```
SELECT * FROM table_name;
```

- Example: Consider the STUDENT table having the following records:

SQL> SELECT * FROM STUDENT;				
REGNO	NAME	COMB	DOB	FEES
1401	RAMESH	PCMC	07-AUG-99	14000
1402	JOHN	PCMB	15-SEP-99	13500
1403	GANESH	PCME	19-AUG-99	16000
1404	MAHESH	PCMC	14-JAN-98	17650
1405	SURESH	PCMB	03-MAR-98	11500
1410	ARUN	PCMC	01-APR-04	13000
1411	SHREYA	CEBA		24000
7 rows selected.				

- Following is an example, which would fetch REGNO, NAME and COMBINATION fields of the customers available in STUDENT table:

```
SQL> SELECT REGNO, NAME, COMBINATION FROM STUDENT;
```

REGNO	NAME	COMB
1401	RAMESH	PCMC
1402	JOHN	PCMB
1403	GANESH	PCME
1404	MAHESH	PCMC
1405	SURESH	PCMB
1410	ARUN	PCMC
1411	SHREYA	CEBA

7 rows selected.

- There may be a situation when you have multiple duplicate records in a table. While fetching such records, it makes more sense to fetch only unique records instead of fetching duplicate records.
- Syntax: The basic syntax of DISTINCT keyword to eliminate duplicate records is as follows:

```
SELECT DISTINCT      column1, column2,... columnN
FROM                  Table_name
WHERE                 [condition]
```

- Example: Consider the STUDENT table having the following records:

```
SQL> select * from student;
```

REGNO	NAME	COMB	DOB	FEES
1401	RAMESH	PCMC	07-AUG-99	14000
1402	JOHN	PCMB	15-SEP-99	13500
1403	GANESH	PCME	19-AUG-99	16000
1404	MAHESH	PCMC	14-JAN-98	17650
1405	SURESH	PCMB	03-MAR-98	11500
1410	ARUN	PCMC	01-APR-04	13000
1411	SHREYA	CEBA		24000

7 rows selected.

- First, let us see how the following SELECT query returns duplicate combination records:

```
SQL> SELECT COMBINATION FROM STUDENT ORDER BY COMBINATION;  
  
COMB  
----  
CEBA  
PCMB  
PCMB  
PCMC  
PCMC  
PCMC  
PCMC  
PCME  
  
7 rows selected.
```

- Now, let us use DISTINCT keyword with the above SELECT query and see the result:

```
SQL> SELECT DISTINCT COMBINATION FROM STUDENT  
ORDER BY COMBINATION;
```

- This would produce the following result where we do not have any duplicate entry:

```
SQL> SELECT DISTINCT COMBINATION FROM STUDENT ORDER BY COMBINATION;  
  
COMB  
----  
CEBA  
PCMB  
PCMC  
PCMC  
PCME
```

✓ **WHERE clause – (Extracting specific rows)**

- The SQL **WHERE** clause is used to specify a condition while fetching the data from single table or joining with multiple tables.
- If the given condition is satisfied then only it returns specific value from the table. You would use WHERE clause to filter the records and fetching only necessary records.

- The WHERE clause is not only used in SELECT statement, but it is also used in UPDATE, DELETE statement, etc.
- Syntax: The basic syntax of SELECT statement with WHERE clause is as follows:

```
SELECT      column1, column2, columnN
FROM        Table_name
WHERE       [condition]
```

- You can specify a condition using comparison or logical operators like >, <, =, LIKE, NOT, etc.
- Following is an example which would fetch REGNO, NAME and FEES fields from the STUDENT table where FEES is greater than 15000:

SQL> SELECT REGNO, NAME, FEES FROM STUDENT WHERE FEES>15000;		
REGNO	NAME	FEES
1403	GANESH	16000
1404	MAHESH	17650
1411	SHREYA	24000

- Following is an example, which would fetch REGNO, NAME and COMBINATION fields from the STUDENT table for a COMBINATION is "PCMC".
- Here, it is important to note that all the strings should be given inside single quotes (' ') where as numeric values should be given without any quote as in above example:

SQL> SELECT REGNO, NAME, COMBINATION FROM STUDENT WHERE COMBINATION='PCMC';		
REGNO	NAME	COMB
1401	RAMESH	PCMC
1404	MAHESH	PCMC
1410	ARUN	PCMC

- The SQL **AND** and **OR** operators are used to combine multiple conditions to narrow data in an SQL statement. These two operators are called *conjunctive operators*.
- These operators provide a means to make multiple comparisons with different operators in the same SQL statement.

✓ The AND Operator:

- The **AND** operator allows the existence of multiple conditions in an SQL statement's WHERE clause.
- Syntax: The basic syntax of AND operator with WHERE clause is as follows:

```
SELECT      column1, column2, columnN
FROM        Table_name
WHERE       [condition1] AND [condition2]...AND [conditionN];
```

- You can combine N number of conditions using AND operator. For an action to be taken by the SQL statement, whether it be a transaction or query, all conditions separated by the AND must be TRUE.
- Example: Consider the STUDENT table having the following records:

REGNO	NAME	COMB	DOB	FEES
1401	RAMESH	PCMC	07-AUG-99	14000
1402	JOHN	PCMB	15-SEP-99	13500
1403	GANESH	PCME	19-AUG-99	16000
1404	MAHESH	PCMC	14-JAN-98	17650
1405	SURESH	PCMB	03-MAR-98	11500
1410	ARUN	PCMC	01-APR-04	13000
1411	SHREYA	CEBA		24000

- Following is an example, which would fetch REGNO, NAME and DOB fields from the STUDENT table where fees is less than 15000 AND combination is "PCMC":

SQL> SELECT REGNO, NAME, DOB FROM STUDENT WHERE FEES<15000 AND COMBINATION='PCMC';				
REGNO	NAME	DOB		
1401	RAMESH	07-AUG-99		
1410	ARUN	01-APR-04		

✓ The OR Operator:

- The OR operator is used to combine multiple conditions in an SQL statement's WHERE clause.
- Syntax: The basic syntax of OR operator with WHERE clause is as follows:

```
SELECT      column1, column2, columnN
FROM        Table_name
WHERE       [condition1] OR [condition2]...OR [conditionN];
```

- You can combine N number of conditions using OR operator. For an action to be taken by the SQL statement, whether it be a transaction or query, only any ONE of the conditions separated by the OR must be TRUE.
- Following is an example, which would fetch REGNO, NAME and DOB fields from the STUDENT table where fees is less than 1500 OR combination is PCMC:

SQL> SELECT REGNO, NAME, DOB FROM STUDENT WHERE FEES<15000 OR COMBINATION='PCMC';				
REGNO	NAME	DOB		
1401	RAMESH	07-AUG-99		
1402	JOHN	15-SEP-99		
1404	MAHESH	14-JAN-98		
1405	SURESH	03-MAR-98		
1410	ARUN	01-APR-04		

✓ ORDER BY – (Sorting the data)

- The SQL **ORDER BY** clause is used to sort the data in ascending or descending order, based on one or more columns. Some database sorts query results in ascending order by default.
- Syntax: The basic syntax of ORDER BY clause is as follows:

```
SELECT      column-list
FROM        Table_name
[WHERE      condition]
[ORDER BY   column1, column2, .. columnN] [ASC | DESC];
```

- You can use more than one column in the ORDER BY clause. Make sure whatever column you are using to sort, that column should be in column-list.
- Example: Consider the STUDENT table having the following records:

REGNO	NAME	COMB	DOB	FEES
1401	RAMESH	PCMC	07-AUG-99	14000
1402	JOHN	PCMB	15-SEP-99	13500
1403	GANESH	PCME	19-AUG-99	16000
1404	MAHESH	PCMC	14-JAN-98	17650
1405	SURESH	PCMB	03-MAR-98	11500
1410	ARUN	PCMC	01-APR-04	13000
1411	SHREYA	CEBA		24000

- Following is an example, which would sort the result in ascending order by NAME:

```
SQL> SELECT * FROM STUDENT ORDER BY NAME;
```

REGNO	NAME	COMB	DOB	FEES
1410	ARUN	PCMC	01-APR-04	13000
1403	GANESH	PCME	19-AUG-99	16000
1402	JOHN	PCMB	15-SEP-99	13500
1404	MAHESH	PCMC	14-JAN-98	17650
1401	RAMESH	PCMC	07-AUG-99	14000
1411	SHREYA	CEBA		24000
1405	SURESH	PCMB	03-MAR-98	11500

7 rows selected.

- Following is an example, which would sort the result in descending order by NAME:

```
SQL> SELECT * FROM STUDENT ORDER BY NAME DESC;
```

REGNO	NAME	COMB	DOB	FEES
1405	SURESH	PCMB	03-MAR-98	11500
1411	SHREYA	CEBA		24000
1401	RAMESH	PCMC	07-AUG-99	14000
1404	MAHESH	PCMC	14-JAN-98	17650
1402	JOHN	PCMB	15-SEP-99	13500
1403	GANESH	PCME	19-AUG-99	16000
1410	ARUN	PCMC	01-APR-04	13000

7 rows selected.

✓ SQL Functions:

- SQL functions are used to manipulate data and return a result.
- SQL provides many built-in functions, which are mainly of two types:

Group Functions – operate on a set of values and return a single result (e.g., SUM, AVG, COUNT).

Scalar Functions – operate on a single value and return a single result (e.g., UPPER, LOWER, ROUND).

- **Group Functions (Aggregate Functions):**

- Functions that act on set of values are called group functions.
- A group functions can takes entire column of data as its arguments and produces a single data item that summarizes the column.
- Following are the SQL group functions.

Function	Description
AVG	Returns average value of “N”, ignoring NULL values
COUNT(expr)	Returns the number of rows where “expr” is not NULL
COUNT(*)	Returns the number of rows in the table including duplicates and those with NULL values
MIN	Returns minimum value of “expr” note: expr stands for expression. An expression can be: A column name (e.g., salary)
MAX	Returns maximum value of “expr”
SUM	Returns sum of values “N”

- **Scalar Functions:**

- Functions that **act on only one value at a time are called scalar functions.**
- We can further classify the functions using the type of data with they are designed to work.

Function	Description
Numeric Functions	Work with numbers. Examples: POWER, ROUND
String Functions	Work with character-based data. Examples: LOWER, UPPER, SUBSTR, LENGTH
Date Functions	Work with Date data types. Example: ADD_MONTHS, LAST_DAY, MONTHS_BETWEEN, NEXT_DAY
Conversion Functions	These functions are used to convert one type of data to another. Example: TO_NUMBER, TO_CHAR, TO_DATE

Consider the EXAMINATION table:

RegNo	Name	CC	Phy	Che	Mat	Cs	Total	City
101	Ajay	C1	98	100	97	99	394	Hassan
102	Banu	C2	38	50	37	49	174	Belur
103	Chandan	C2	100	100	97	99	396	Mysuru
104	John	C3	78	80	67	79	304	Alur
105	Kaleem	C1	88	80	91	79	338	Hassan
106	Raheem	C2	100	98	97	79	374	Hassan
107	Sanjay	C3	47	60	56	78	241	Alur
108	Tarun	C3	33	34	77	28	172	Arasikere
109	Uday	C2	100	98	97	79	374	Hassan
110	Venki	C3	47	60	56	78	241	Belur

✓ COUNT () Function:

- This function is used to count the number of values in a column.
- COUNT (*) is used to count the number of rows in the table including duplicates and those with NULL values.
- Example 1:
 - SELECT COUNT (*) FROM EXAMINATION;
 - The above query returns 10.
- Example 2:
 - SELECT COUNT (RegNo) FROM EXAMINATION WHERE CC = "C3";
 - The above query returns 4.

✓ AVG () Function:

- This function is used to find the average of the values in a numeric column.
- Example 1:
 - SELECT AVG (Cs) FROM EXAMINATION;
 - The above query returns 74.7

✓ SUM () Function:

- This function is used to find the sum of the values in a numeric column.
- Example:
 - SELECT SUM (Phy) FROM EXAMINATION;
 - The above query returns 729

✓ MAX () Function:

- This function is used to find the maximum values in a column.
- Example:
 - SELECT MAX (Phy) FROM EXAMINATION;
 - The above query returns 100

✓ MIN () Function:

- This function is used to find the minimum values in a column.
- Example:
 - SELECT MIN (Phy) FROM EXAMINATION;
 - The above query returns 33

✓ GROUP BY (Grouping Result)

- The SQL **GROUP BY** clause is used in collaboration with the SELECT statement to arrange identical data into groups.
- The GROUP BY clause follows the WHERE clause in a SELECT statement and precedes the ORDER BY clause.
- **Syntax:** The basic syntax of GROUP BY clause is given below.

```
SELECT      column1, column2
FROM        Table_name
WHERE       [ conditions ]
GROUP BY    column1, column2
ORDER BY    column1, column2
```

- Example 1: To find the number of students in each college.

```
SELECT      CC, COUNT (CC)
FROM        EXAMINATION
GROUP BY    CC;
```

- Example 2: To find the number of students, sum, average, maximum, minimum marks in computer science from each city.

```
SELECT      City, COUNT (City), SUM (Cs), AVG (Cs), MAX (Cs), MIN (Cs)
FROM        EXAMINATION
GROUP BY    City;
```

➤ SQL CONSTRAINTS:

- Constraints are the rules enforced on data columns on table.
- These are limiting the type of data that can go into a table.
- This ensures the accuracy and reliability of the data into the database.
- SQL allows two types of constraints.
 - **Column level constraints:** These constraints are defined along with the column definition when creating or altering a table structure. These constraints apply only to individual columns.
 - **Table level constraints:** These constraints are defined after all the table columns when creating or altering a table structure. Apply to multiple columns together (one or more).

Following are the commonly used constraints available in SQL.

Constraints	Description
NOT NULL	Ensures that a column cannot have NULL value
UNIQUE	Ensures that all values in column are different
PRIMARY KEY	Uniquely identified each row in a database table.
FOREIGN KEY	Uniquely identified each row in any other database table
DEFAULT	Provides a default value for a column when none is specified
CHECK	Ensures that all values in a column satisfy certain condition.

✓ NOT NULL Constraint:

- By default column can hold NULL values.
- When a column is defined as NOT NULL then the column becomes a mandatory column.
- It implies that a value must be entered into the column if the row is to be inserted for storage in the table.

- Example: Consider the following CREATE TABLE command creates a new table called PRODUCT and add six columns, two which PID and Description specify not to accept NULLs.

```
CREATE TABLE PRODUCT
(
    PID                CHAR (4)          NOT NULL,
    Description        VARCHAR2 (25), NOT NULL
    CompanyId         CHAR (10),
    DOM               DATE,
    Type              CHAR (10),
    Price             NUMBER (10,2)
);
```

✓ UNIQUE Constraints:

- This constraint ensures that no rows have the same value in the specified column(s). A table must have many unique keys.
- Example: UNIQUE constraint applied on PID of PRODUCT table ensures that no rows have the same PID value, as shown below

```
CREATE TABLE PRODUCT
(
    PID                CHAR (4)          NOT NULL UNIQUE,
    Description        VARCHAR2 (25), NOT NULL
    CompanyId         CHAR (10),
    DOM               DATE,
    Type              CHAR (10),
    Price             NUMBER (10,2)
);
```

✓ PRIMARY KEY Constraints:

- A primary key is a field which uniquely identifies each row in a database table. A primary key in a table has special attributes:
- By default this column is NOT NULL. It defines the column as a mandatory column i.e. the column cannot be left blank.
- The data held in this column must be unique.
- Example:

```
CREATE TABLE PRODUCT
(
    PID                CHAR (4)          PRIMARY KEY,
    Description        VARCHAR2 (25) NOT NULL
    CompanyId         CHAR (10),
    DOM               DATE,
    Type              CHAR (10),
    Price             NUMBER (10,2)
);
```

✓ FOREIGN KEY Constraint:

- A FOREIGN KEY is used to link two tables together.
- A foreign key is a column whose values are derived from the PRIMARY KEY of some other table.

Example:

```
create table person(pid int not null primary key, name varchar(30),age int);
```

```
create table personorder(orderid int not null primary key, ordernumber int, pid int,foreign key(pid) references person(pid));
```

✓ DEFAULT Constraints:

- A default value can be specified for a column using the DEFAULT clause.
- The DEFAULT constraint provides a default value to a column when the INSERT INTO command does not provide a specific value.
- Example:

```
CREATE TABLE PRODUCT
(
    PID          CHAR (4)          PRIMARY KEY,
    Description   VARCHAR2 (25), NOT NULL
    CompanyId    CHAR (10),
    DOM          DATE,
    Type         CHAR (10),
    Price        NUMBER (10, 2) DEFALUT 1000.00
);
```

✓ CHECK Constraints:

- The CHECK Constraint is used to establish a TRUE/FALSE condition that is applied to the data placed in a column.
- If a value does not meet the condition, it cannot be placed in the column.
- Example:

```
CREATE TABLE PRODUCT
(
    PID  CHAR (4) CHECK (PID LIKE 'P%'),
    Description   VARCHAR2 (25),
    CompanyId    CHAR (10),
    DOM          DATE,
    Type         CHAR (10),
    Price        NUMBER (10, 2) CHECK (Price>0)
);
```

i.e 'P%' means: first character must be 'P'

✓ TABLE Constraints:

- When a constraint is applied to a group of columns of the table, it is called a table constraint.

- Column constraint is defined along with the end of the column.
- Table constraints are defined at the end of the table.
- Example:

```
CREATE TABLE PRODUCT
(
    PID          CHAR (4)          NOT NULL,
    Description   VARCHAR2 (25) NOT NULL,
    CompanyId    CHAR (10),
    DOM          DATE,
    Type         CHAR (10),
    Price        NUMBER (10, 2),
    PRIMARY KEY (PID, Description); -- Table constraint (composite primary
key)
```

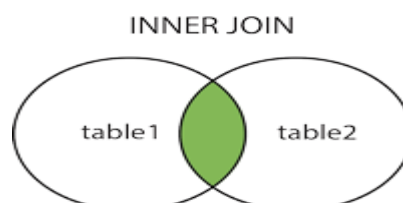
Joins

- The SQL **Joins** clause are used to fetch/retrieve data from two or more tables based on the join condition which is specified in the WHERE condition.
- Basically data tables are related to each other with keys. We can use these keys relationship in SQL joins.

SQL Join Types:

- There are different types of joins available in SQL:
 1. **INNER JOIN**
 2. **LEFT JOIN**
 3. **RIGHT JOIN**
 4. **FULL JOIN**

INNER JOIN: In SQL, INNER JOIN selects records that have matching values in both tables as long as the condition is satisfied. It returns the combination of all rows from both the tables where the condition satisfies.



Example

Table: Students

stu_id	name	department	year
1	Ram	CS	1
2	Sita	IT	2
3	Hari	CS	1
4	Gita	SE	3

Table: Marks

mark_id	stu_id	subject	marks
11	1	DBMS	80
12	1	DSA	85
13	3	DBMS	90
14	5	DSA	70

Query

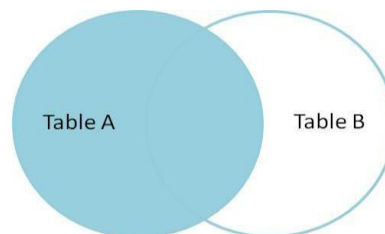
```
SELECT s.stu_id, s.name, s.department, s.year, m.subject, m.marks
FROM Students s
INNER JOIN Marks m
ON s.stu_id = m.stu_id;
```

Result

stu_id	name	department	year	subject	marks
1	Ram	CS	1	DBMS	80
1	Ram	CS	1	DSA	85
3	Hari	CS	1	DBMS	90

2. LEFT JOIN

The SQL left join returns all the values from left table and the matching values from the right table. If there is no matching join value, it will return NULL.

**Query**

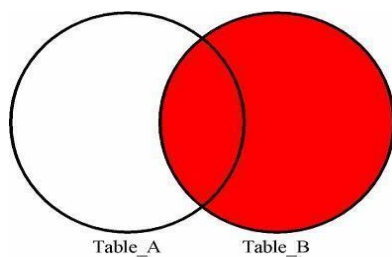
```
SELECT s.stu_id, s.name, s.department, s.year, m.subject, m.marks
FROM Students s
LEFT JOIN Marks m
ON s.stu_id = m.stu_id;
```

Result

stu_id	name	department	year	subject	marks
1	Ram	CS	1	DBMS	80
1	Ram	CS	1	DSA	85
2	Sita	IT	2	NULL	NULL
3	Hari	CS	1	DBMS	90
4	Gita	SE	3	NULL	NULL

3. RIGHT JOIN

In SQL, RIGHT JOIN returns all the values from the values from the rows of right table and the matched values from the left table. If there is no matching in both tables, it will return NULL.

**Query**

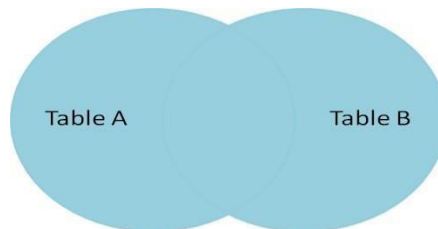
```
SELECT s.stu_id, s.name, s.department, s.year, m.subject, m.marks
FROM Students s
RIGHT JOIN Marks m
ON s.stu_id = m.stu_id;
```

Result

stu_id	name	department	year	subject	marks
1	Ram	CS	1	DBMS	80
1	Ram	CS	1	DSA	85
3	Hari	CS	1	DBMS	90
NULL	NULL	NULL	NULL	DSA	70

4. FULL OUTER JOIN

In SQL, FULL JOIN is the result of a combination of both left and right outer join. Join tables have all the records from both tables. It puts NULL on the place of matches not found.



Query

```
SELECT s.stu_id, s.name, s.department, s.year, m.subject, m.marks
FROM Students s
FULL OUTER JOIN Marks m
ON s.stu_id = m.stu_id;
```

Result

stu_id	name	department	year	subject	marks
1	Ram	CS	1	DBMS	80
1	Ram	CS	1	DSA	85
2	Sita	IT	2	NULL	NULL
3	Hari	CS	1	DBMS	90
4	Gita	SE	3	NULL	NULL
NULL	NULL	NULL	NULL	DSA	70

Query By Example (QBE)

Query By Example (QBE) is a database query method that allows users to search for data by filling out a template or form, instead of writing complex SQL queries. It provides a visual and user-friendly interface, making it easier for non-technical users to retrieve data from a database.

How QBE Works

- Users enter example values or conditions into a special query grid (or form) that represents the database table structure.
- The database system automatically translates these inputs into SQL commands and executes the query.
- The results matching the given example are displayed.

Example Scenario

Imagine a **Students** table with columns: `Student_ID`, `Name`, `Age`, `Grade`.

- If a user wants to find students with **Grade = 'A'**, they enter 'A' in the Grade column of the QBE grid.
- The system converts it into an SQL query like:

```
SELECT * FROM Students WHERE Grade = 'A';
```

- The database retrieves and displays matching records.

Advantages of QBE

- ✓ **User-Friendly** – No need to memorize SQL syntax.
- ✓ **Automatic SQL Generation** – The system creates SQL queries based on input.
- ✓ **Faster Query Building** – Easier and quicker than manual SQL writing.

Disadvantages of QBE

- ✓ **Limited Flexibility** – Not suitable for very complex queries.
- ✓ **Less Control** – Advanced users may prefer SQL for more precise queries.

Important Questions

Five Marks Question:

1. Explain various group functions in SQL.
2. What is data definition language? Explain SELECT and UPDATE command.
3. Describe any five logical operators available in SQL.
4. What is SQL? Explain the different SQL commands.
5. What is the purpose of CREATE command? Write the syntax and example of CREATE command.
6. Explain SELECT statement with syntax and with minimum 3 examples.
7. Explain 5 variations of SELECT command.
8. What are SQL constraints? Explain any two constraints.
